



第七章 指针



函数空间的阻隔

例：设计子函数交换两变量的值。

```
int swap(int x,int y);  
int main(){  
    int a,b;  
    scanf("%d%d",&a,&b);  
    swap( a, b );  
    printf("%d,%d\n",a,b);  
}
```

```
int swap(int x,int y){  
    int temp;  
    temp = x;  
    x = y;  
    y = temp;  
    return 0;  
}
```

数组名传参的效果

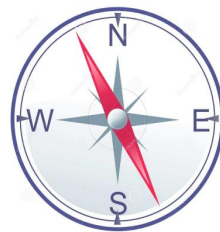
```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
void reverseSeq(int a[], int n);
int main()
{
    int
a[SIZE]={1,2,6,7,8,9,12,25,27,36};
    reverseSeq(a,SIZE);
    printArray(a,SIZE);
    return 0;
}
```

```
void reverseSeq(int a[], int n)
{ //将整数数组中的元素逆序存放

    int i,tmp;
    for(i=0; i<(n/2-1); i++){
        tmp = a[i];
        a[i] = a[n-1-i];
        a[n-1-i] = tmp;
    }
}
```

用数组名实现变量交换子程序

```
void swap( int a[], int b[]){  
    int tmp;  
    tmp=a[0];  
    a[0] = b[0];  
    b[0] = tmp;  
}
```



7.1 指针~概念与含义

一、变量的地址

```
int i; char ch; float f;
```

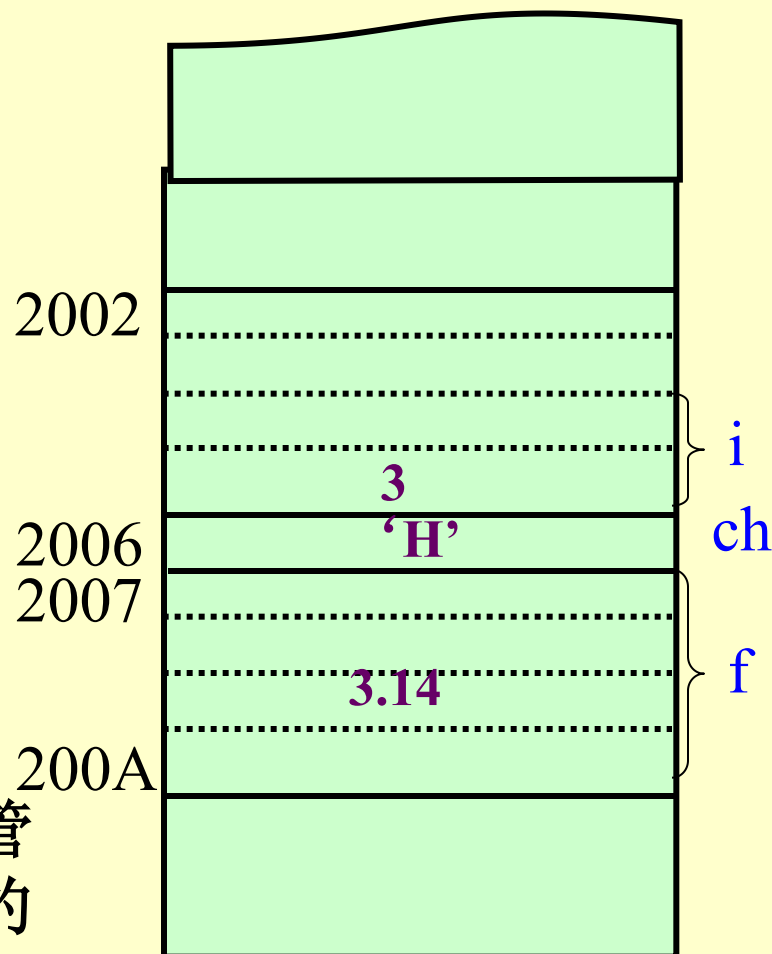
```
i=3; ch='H'; f=3.14;
```

变量被保存在内存中，
因此有地址。

变量地址——系统分
配给变量的内存单元
的起始地址

如：i的地址是2002

变量一旦被分配地址，尽管
它的内容可以被修改，它的
地址将永远不会改变



7.1 指针~概念与含义

二、变量的访问

(1) 直接访问——在变量作用域内部,直接利用变量名,获取变量地址进行访问.

1) `scanf("% d",&i)`的执行过程是这样的:

先找到变量*i*的起始地址2002; 然后将键盘输入的值送到内存单元2002~2005中。

2) `printf(" i=%d", i)`的执行过程, 与`scanf()`很相似: 首先找到变量*i*的起始地址2002, 然后从内存单元2002~2005中取出值, 将它输出。

7.1 指针~概念与含义

(2) 间接访问—在变量作用域之外,可以通过指针(pointer)来访问.

C程序的**指针变量**, 用来存放所指变量的内存起始地址。

数据类型 * 指针变量名;

例：设计子函数交换两变量的值。

```
void swap( int * p1, int * p2);
```

```
int main(){
```

```
    int a,b;
```

```
    scanf("%d%d",&a,&b);
```

```
    swap( &a, &b );
```

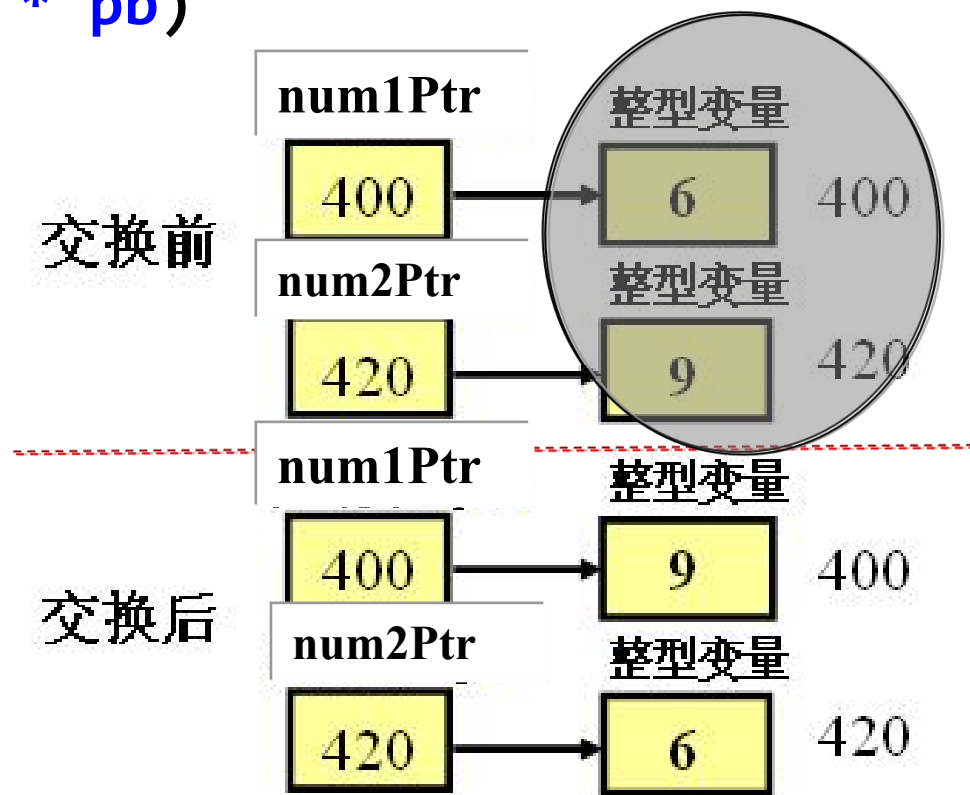
```
    printf("%d,%d\n",a,b);
```

```
}
```


指针访问运算符*

```
void swap(int * pa, int * pb)
{
    int tmp;
    tmp=*pa;
    *pa = *pb;
    *pb = tmp;
}
```

表达式中 ***指针变量**
表示按地址访问的运算。



7.2 指针和数组

C语言中指针和数组有着密切关系:

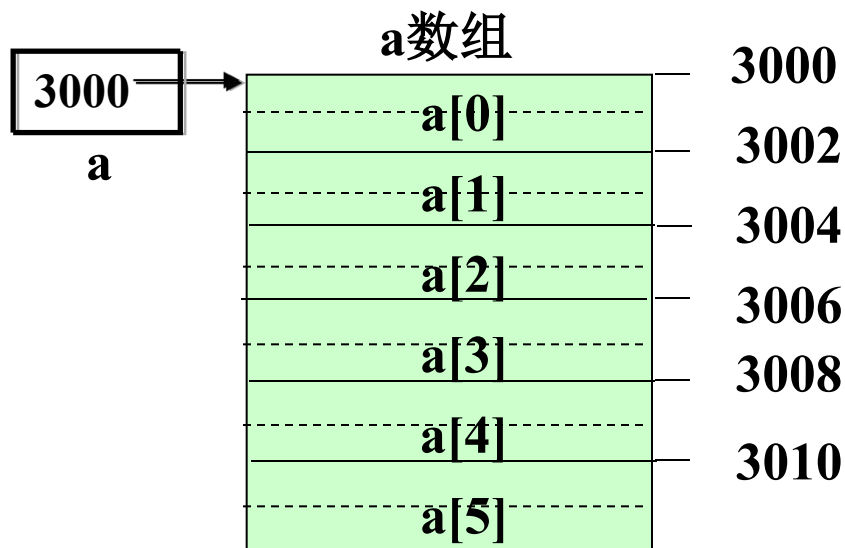
1. 一维数组名可看作是一个指向数组中下标为0的元素的指针常量。

例:

```
int a [6];
```

```
a[0]=3; //数组下标访问法
```

```
*(a+3)=5; //指针表示法
```





例7.5 通过指针输出 a 数组元素的两种方法。

```
void outArray1(int  
a[],int n)  
{   int i;  
    for(i=0;i<n;i++){  
        if(i==n-1)  
            printf("%d\n",a[i]);  
        else  
            printf("%d,",a[i]);  
    }  
}
```

```
void outArray2(int * pa,int  
n)  
{   int i;  
    for(i=0; i<n; i++){  
        if(i==n-1)  
            printf("%d\n",*(pa+i));  
        else  
            printf("%d,",*(pa+i));  
    }  
}
```

```
#define SIZE 10  
  
void outArray1(int a[],int n);  
void outArray2(int * pa,int n);  
int main()  
{  
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};  
    outArray1(a,SIZE);  
    outArray2(a,SIZE);  
  
    return 0;  
}
```

例7.6 通过指针对 a 数组元素冒泡排序

```
void swap(int *, int *);/*函数原型*/
void bubbleSort(int * pa, const int n)
{
    int i, j;

    for (i=1;i<=n-1; i++)
        for (j=0;j<=n-1-i; j++)
            if ( pa[j] > pa[j+1] )
                swap(pa+j, pa+j+1);
}
```

```
#define SIZE 10
void swap(int *, int *);/*函数原型*/
void outArray2(int * pa,int n);
void bubbleSort(int * pa, const int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    bubbleSort(a,SIZE);
    outArray2(a,SIZE);
    return 0;
}
```



例7.7 将数组 a 中 n 个整数按相反顺序存放。

```
void inverse(int * pa, int n)
{
    int i;
    for(i=0; i<(n/2-1); i++)
        swap(pa+i, pa+n-1-i);
}
```

```
#define SIZE 10
void swap(int *, int *);/*函数原型*/
void inverse(int *,int);
void outArray2(int *,int);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    inverse(a,SIZE);
    outArray2(a,SIZE);
    return 0;
}
```


7.2 指针和数组

2. 二维数组名是指向数组第0行的指针常量。 二级指针

例：

```
int m[3][4];
```

```
m[1][2]=3; //数组下标访问法
```

$*(m+i)$ 等同 $a[i]$, 仍是指针

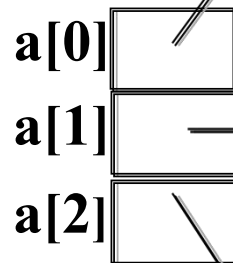
```
 $*(*(m+i)+j)=5;$ 
```

//二级指针表示, 等同 $a[i][j]$

二级指针



一级指针



a[0][0]	
a[0][1]	
a[0][2]	
a[0][3]	
a[1][0]	
a[1][1]	
a[1][2]	//////
a[1][3]	
a[2][0]	
a[2][1]	
a[2][2]	
a[2][3]	

二维数组的指针访问形式

```
int main(){
    int n,m[3][3]={1,2,3,4,5,6,7,8,9};
    int rows=3,cols=3;
    for (int i=0; i<rows; i++){
        for(int j=0; j<cols; j++)
            printf("%d\t",*(*(m+i)+j));
        printf("\n");
    }
}
```

错误示例：子函数不知道列数

```
void printMatrixEr(const int **pm, int rows,int
cols)
{
    int i, j;
    for (i=0; i<rows; i++){
        for(j=0; j<cols; j++)
            printf("%d\t", *(*(pm+i)+j));
        printf("\n");
    }
}
```

二维数组的指针传参

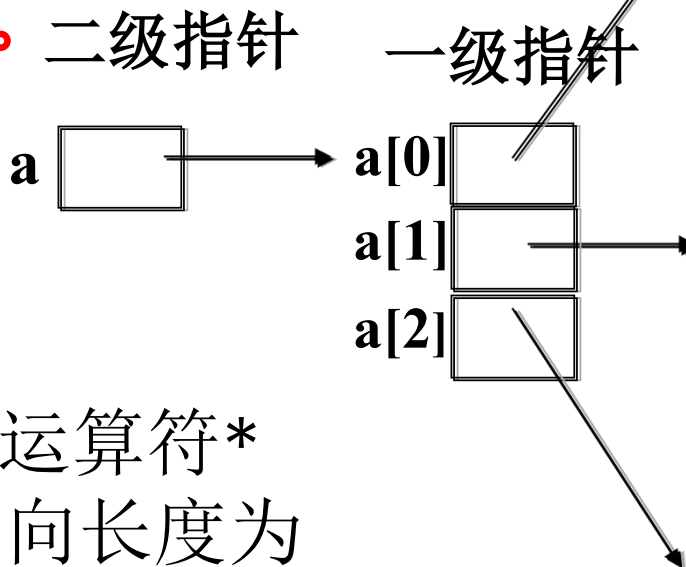
**二维数组名是指向一维数组
指针的指针常量。二级指针**

例：

```
int m[3][4];
```

```
int (*pm)[4];
```

因[]优先级比指针运算符*高,加括号;pm是指向长度为4的一维数组的数组名的指针,所以和二维数组名等价。



a[0][0]	
a[0][1]	
a[0][2]	
a[0][3]	
a[1][0]	
a[1][1]	
a[1][2]	////
a[1][3]	
a[2][0]	
a[2][1]	
a[2][2]	
a[2][3]	

二维数组的指针传参

```
#define COLS 3
void printMatrix(const int (*pm)[COLS], int
rows,int cols)
{
    int i, j;
    for (i=0; i<rows; i++){
        for(j=0; j<cols; j++)
            printf("%d\t",*(*(pm+i)+j));

        printf("\n");
    }
}
```

不带列数的二维数组指针传参

➤ 定义一个函数，用于输出各种列数的二维数组

```
int m[3][4]; printMatrix( *m, 3, 4);
```

```
void printMatrix(const int * pm, int rows, int  
cols)
```

```
{
```

```
    int i, j;
```

```
    for (i=0; i<rows; i++){
```

```
        for(j=0; j<cols; j++)
```

```
            printf("%d\t", pm[i*cols+j]);
```

```
        printf("\n");
```

```
    }
```

```
}
```

一级指针做形参，无须给列数。

用指针计算matrix[i][j]地址进行
间接访问,这里不能用pm[i][j]

不带列数的二维数组指针传参(例2)

➤ 定义一个函数，用于输入二维数组

```
int m[3][4]; inputMatrix( *m, 3, 4);
```

```
void inputMatrix(int * pm, int rows,int cols)
{
    int i, j;
    for (i=0; i<rows; i++){
        for(j=0; j<cols; j++)
            scanf("%d",pm+i*cols+j);

        printf("\n");
    }
}
```

一级指针做形参，无须给列数。

```
#include <stdio.h>
#define COLS 5
#define ROWS 5
void printMatrix(int *,int,int);
void inputMatrix(int *,int,int);
int main(void){
    int matrix[ROWS][COLS]={0};
    inputMatrix(matrix[0],ROWS,COLS);
    printMatrix(matrix[0],ROWS,COLS);
}
```


对指针使用const限定符

传递指针的4种情况：

1. **int * iPtr ;** /*指向非常量数据的非常量指针。i和iPtr皆能被修改*/
2. **int * const iPtr;** /*指向非常量数据的指针常量。iPtr不能被修改，只能指向i，但iPtr指向的变量可以被修改*/
3. **const int * iPtr ;** /*指向常量数据的非常量指针。iPtr可以修改，即可以指向其他变量；但iPtr指向的变量不能被修改*/
4. **const int * const iPtr;** /*指向常量数据的常量指针。iPtr和iPtr指向的变量皆不能被修改*/

根据实际需要来使用const限定符

库函数malloc()

- 用法:

```
#include<stdlib.h>
```

```
void * malloc(unsigned size);
```

- 功能：在内存的堆区分配size个字节的连续空间。
- 返回值：若申请成功，则返回新分配内存块的起始地址；否则，返回NULL。

malloc()函数的返回值是一个无类型指针，它可以指向任何类型的数据。

7.3 动态分配数组

- 在程序运行过程中，数组的大小是不能改变的。这种数组称为**静态数组**。静态数组的缺点是：对于事先无法准确估计数据量的情况，无法做到既满足处理需要，又不浪费内存空间。

```
#define SIZE 1000
```

```
int a[SIZE];
```

- 但有时我们并不知道我们需要多大的数组，直到程序开始运行。我们最好能做到以下情形：

```
int a[实际需要的大小];
```

C99允许动态定义数组`int a[n];`

什么时候需要用动态分配数组？

1、当我们需要在子函数中定义数组，并在此函数返回后，仍能保存此数组不被销毁时。

参考例 `char* allocStrFromHeap(int len);`

2、当一个数组的大小，会在程序运行过程中发生变化，而我们又不愿意浪费空间时。

参考同上例

- 数组内存的静态分配和动态分配的区别主要是两个：
 - 一是时间不同。静态分配发生在程序编译和连接的时候。动态分配则发生在程序调入和执行的时候。
 - 二是空间不同。
 - 栈分配：不带static修饰且在函数内定义的数组空间都在栈区的函数栈中，随着函数返回就自动释放；
 - 堆分配：堆都是动态分配的，没有静态分配的堆。由函数malloc（包括calloc，realloc）进行分配，必须由程序自己释放（free函数）。



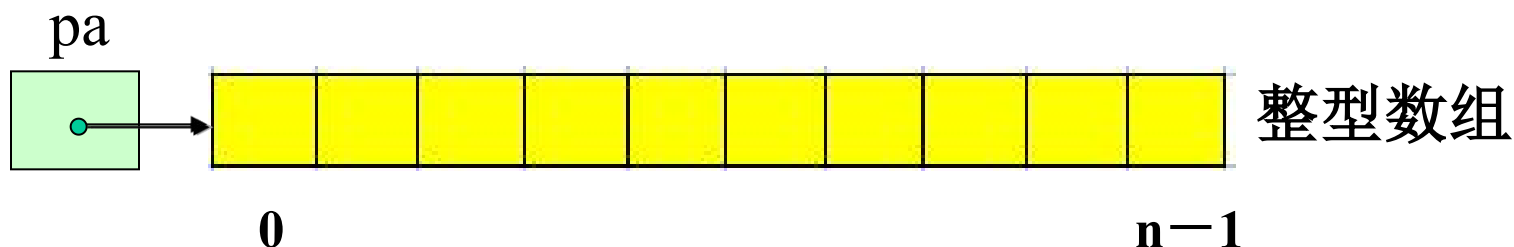
内存

7.3 动态数组

- 我们可以定义一个指针，然后调用**内存申请库函数**在堆区分配一片连续的内存，并将内存的起始地址保存在这个指针中，这就建立了一个**动态数组**。然后通过指针可以访问动态数组的各个元素。

```
int * pa;
```

```
pa = (int *)malloc( sizeof(int) * n );
```





/*栈内存分配*/

```
int*
allocStrFromStack(int
len)
{
    if ( len <= 0 )
        return NULL;
    int a[len];
    return a;
}
```

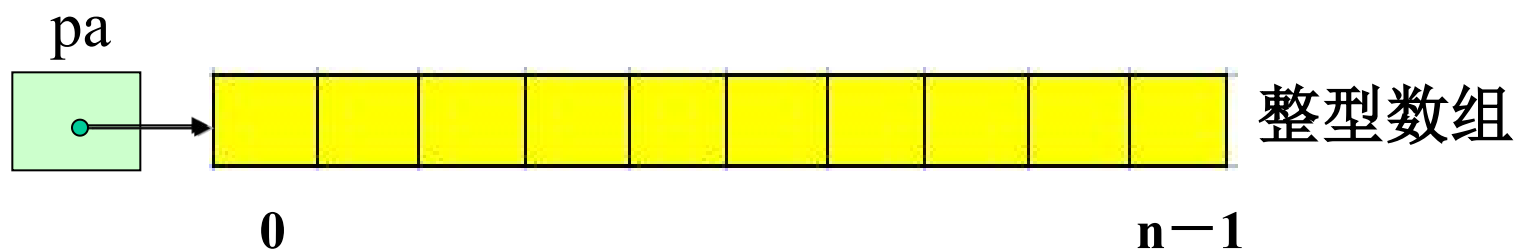
/*堆内存分配*/

```
int* allocStrFromHeap(int
len){
    if ( len <= 0 )
        return NULL;
    else return
(int*)malloc(len*sizeof(int)
);
}
```



```
#include <stdio.h>
void outArray1(int a[],int n);
int* allocStrFromStack(int);
int* allocStrFromHeap(int);
int main()
{
    int n,len;
    int * p1, * p2;
    scanf("%d",&n);
    p1=allocStrFromStack(n);
    outArray1(p1,n); //非法访问已经释放的函数栈内数组
    p2=allocStrFromHeap(n);
    len=createArrayTail(p2,n);
    outArray1(p2,len);
    free(p2);
    return 0;
}
```

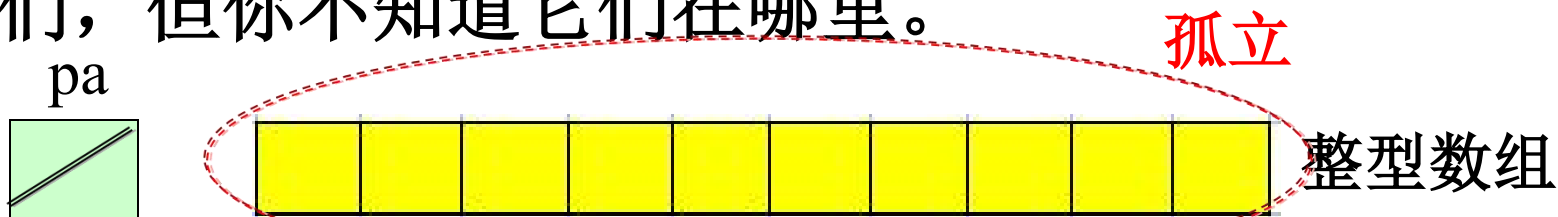
二、如何访问动态数组元素



```
int * pa;  
pa= (int *)malloc(sizeof(int)*n);  
for(i=0; i<n; i++) scanf("%d", pa+i);  
for(i=0; i<n; i++) printf("%d",pa[i]);  
/*或者printf("%d", *(pa+i));*/
```

7.3 动态数组

- 三、释放动态数组占用的存储空间
- 如果你丢失了指针，这些区域将变成不能及时释放的内存块。堆管理器认为你在继续使用它们，但你不知道它们在哪里。



- 为了避免出现不能及时释放的内存块，你应该在不用时释放由malloc申请的内存。

free(pa);

库函数free()

- 用法:

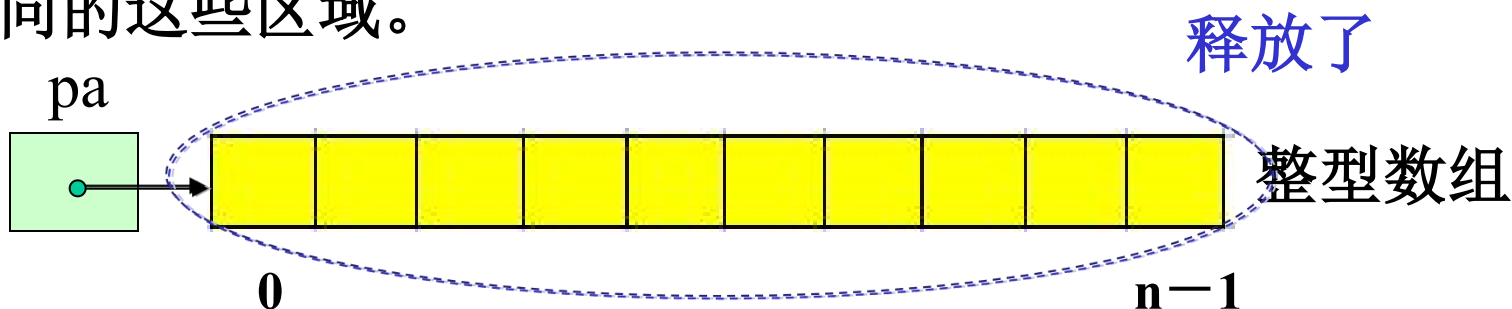
```
#include<stdlib.h>
```

```
void free(void * ptr);
```

- 功能: 释放由ptr指向的内存块 (ptr是调用malloc() 函数的返回值)。
- 返回值: 无。

四、释放了内存后，将指针`pa`赋值为`NULL`，防止产生“野指针”。

- 当你释放了内存区域，堆管理器重新收回这些区域，但你的指针仍然指向堆区域，但你无权再使用指针指向的这些区域。



```
free(pa);
```

```
pa=NULL;
```

```
void *realloc(void *ptr, unsigned int size);
```

功能：重新定义所开辟内存空间的大小。其中ptr所指的内存空间是用内存分配函数已开辟的，size为新的空间大小，其值可比原来大或小。函数返回新存储区的起始地址（该地址可能与以前的地址不同）。

例如：

```
float * pfa=(float *)malloc(10*sizeof(float));
```

```
pfa=(float *)realloc(pfa,16*sizeof(float));
```

将原先开辟的10个小数空间调整为16个小数空间。

7.3 动态数组应用举例

```
#include<stdlib.h>
#include<stdio.h>
#define SIZE 10
void outArray2(int *,int);
int* allocStrFromHeap(int);
int main(){
    int * pa,buf[SIZE],n,len,i,cmd;
    n=createArrayTail(buf,SIZE);
    pa=allocStrFromHeap(n);
    for (i=0;i<n;i++)
        pa[i]=buf[i];
    outArray2(pa,n);
    len = n;
```

```
do{
    n=createArrayTail(buf,SIZE);
    pa=(int *)realloc(pa,sizeof(int)*(len+n));
    for (i=0;i<n;i++)
        pa[len+i]=buf[i];
    len += n;
    outArray2(pa,len);
    printf("Enter 1 continue to read datas,otherwise
enter 0:");
    scanf("%d",&cmd);
}while (cmd==1);
free(pa);
return 0;
}
```

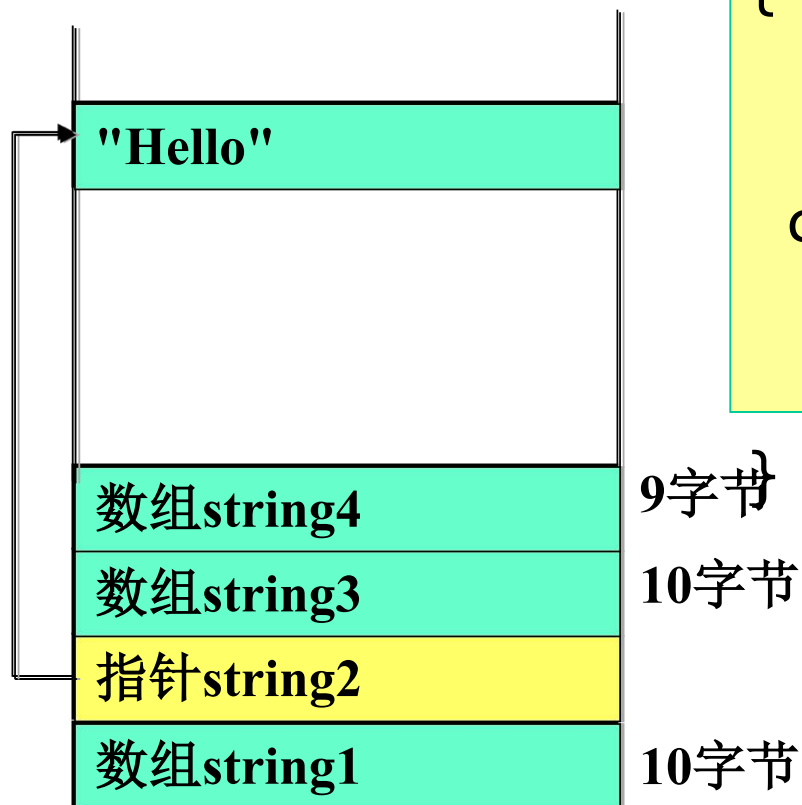

尾插法创建数组子函数

//接收键盘输入的任意个整数,直到换行符结束.

```
int createArrayTail(int a[], int size){
    int i,num; char ch=0;
    printf("Enter integers by a line to create
array.\n");
    for(i=0; i<size && '\n'!=ch;i++){
        scanf("%d",&num);
        a[i]=num;
        ch=getchar();
    }
    return i;
}
```

7.2 指针和字符串常量

在内存的常量区里保存"Hello"



```
int main()
{
    char string1[10],
        string3[10];
    const char * string2 = "Hello";
    char string4[] = "Good Bye";
    .....
```

8字节 (sizeof(char *)值为8) 指针的字节数和计算机字长一致.

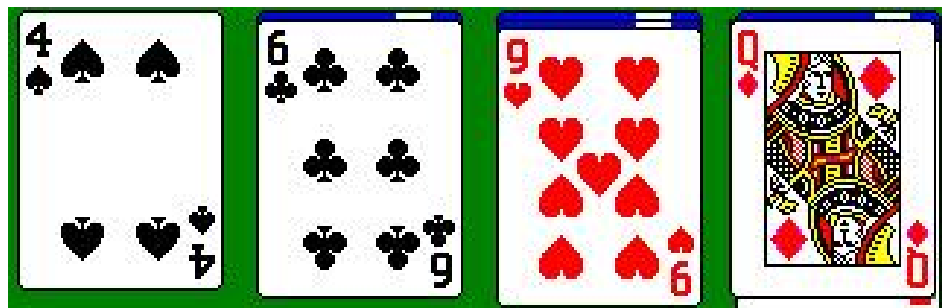
字符常量指针 不是 字符数组！

```
int main()
{
    char * p1="I am a teacher.";
    const char * p2="you are a student.";
    printf("string a=%s\nstring b=%s\n",p1,p2);
    printf("copy string a to string b:\n");
    strcpy(p1,p2); //error
    printf("\nstring a=%s\nstring b=%s\n",p1,p2);
    if (strcmp(p1,p2)==0)
        printf("p1==p2\n");
    else printf("p1!=p2\n");
}
```

实例研究一：（模拟发牌和洗牌）

■ 问题描述

每一付牌共有52张（不考虑大小王），
有13种面值，每1种面值 有4种花色。



■ 思考：

1. 计算机如何表示？（抽象）
2. 如何存储？
3. 如何模拟洗牌与发牌？



麻烦的显示牌

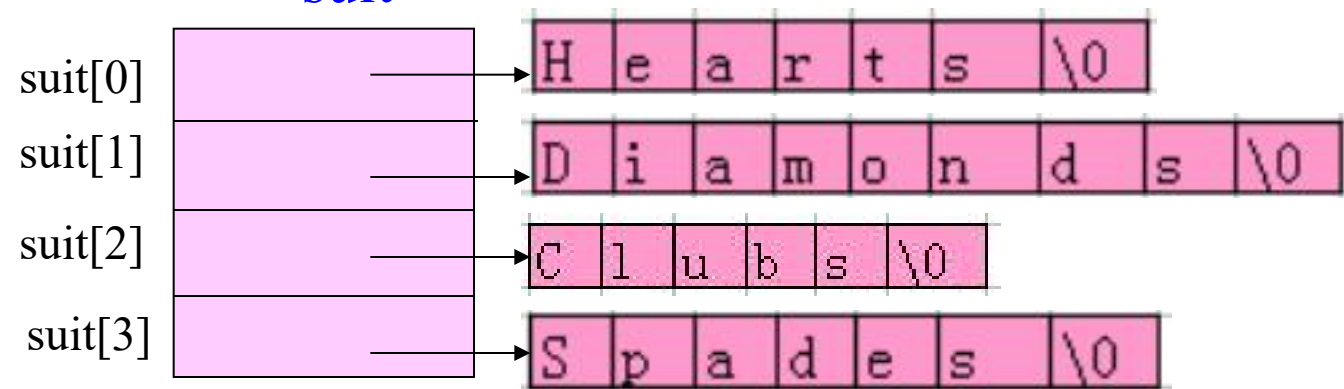
```
void outputCard(int i,int
row,int col)
{
    printf("第%d张牌: ",i);
    switch (col){
        case
0:printf("Acer");break;
        case
1:printf("Deuce");break;
        case
2:printf("Three");break;
        case
3:printf("Four");break;
        case
4:printf("Five");break;
        case
5:printf("Six");break;
        case
6:printf("Seven");break;
        case
7:printf("Eight");break;
```

```
case 8:printf("Nine");break;
        case 9:printf("Ten");break;
        case 10:printf("Jack");break;
        case
11:printf("Queen");break;
        case 12:printf("King");break;
        default:printf("无效的面
值 !");
    }
    printf(" of ");
    switch (row){
        case
0:printf("Hearts");break;
        case
1:printf("diamonds");break;
        case 2:printf("Clubs");break;
        case
3:printf("Spades");break;
        default:printf("无效的花
色 !");
    }
    printf("\n");
```

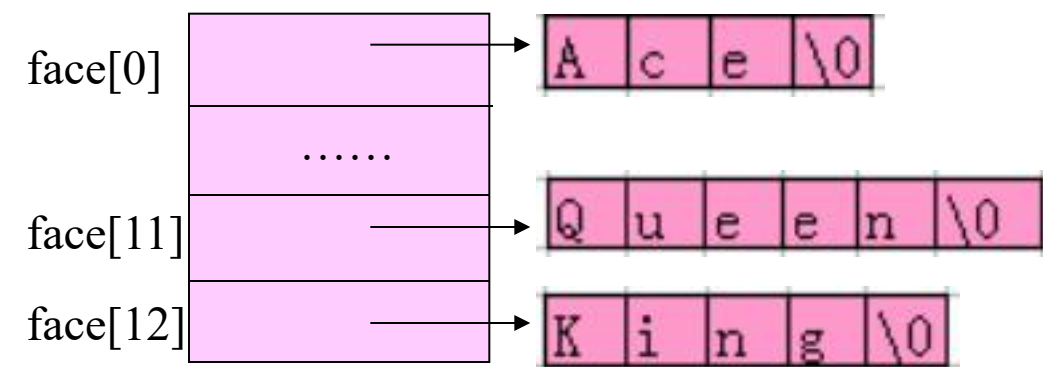
- 也可采用指针数组输出字符串的方式

定义两个指针数组suit和face，分别用来翻译deck的行下标到花色的对应、列下表到面值的对应。

suit



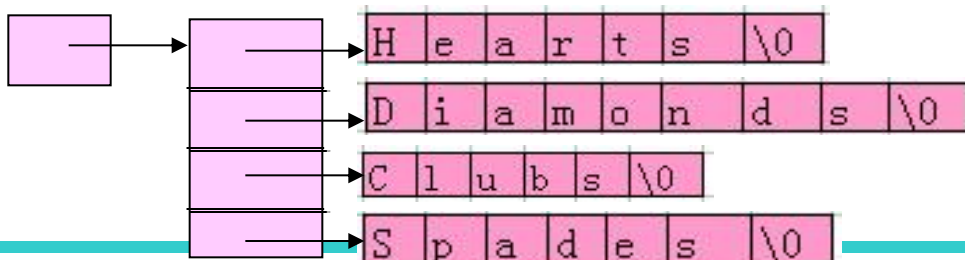
face



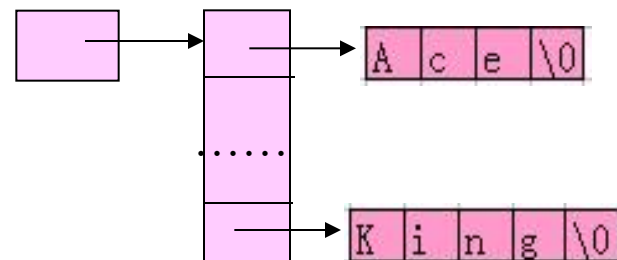
如果deck中第0行第0列的值是10，表示第10张牌是heart A



suit



face



/*发牌程序*/

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void shuffle (int[][13]);
void deal(const int[][13],const char *[],const char *[]);
int main(void)
{
    const char * suit[4]={"Hearts", "diamonds", "Clubs",
    "Spades"};
    const char * face[13]={"Acer", "Deuce", "Three", "Four",
    "Five", "Six", "Seven",
    "Eight", "Nine", "Ten",
    "Jack", "Queen", "King"};
    int deck[4][13]={0};
    srand((unsigned)time(NULL));
    shuffle(deck);//洗牌
    deal(deck, face, suit);//发牌
    return 0;
```

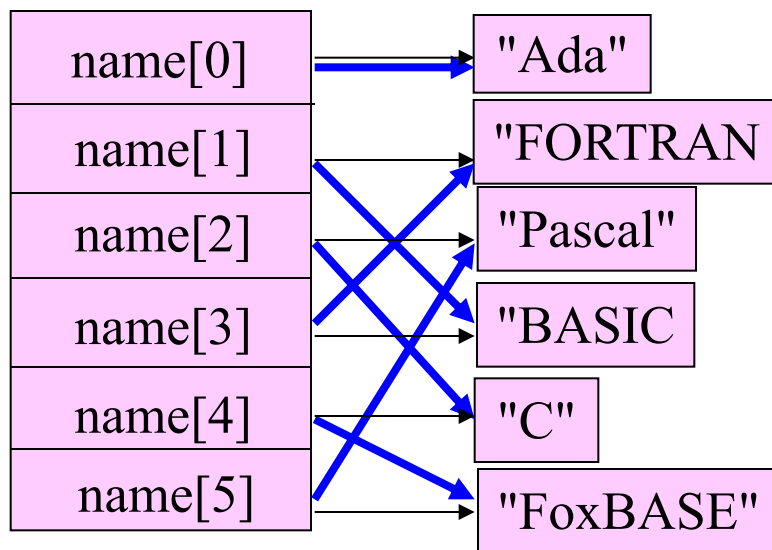


```
void deal (const int *wdeck, char *wface, char * wsuit) //发牌
{
    int card,row,column,found;

    for (card = 1;card <= 52;card++){
        //遍历二维数组，查找值为card的元素，输出对应的花色和面值
        found = 0;
        for (row=0; row <= 3 && !found; row++)
            for (column=0; column <= 12 && !found; column++)
                if (*(wdeck+row*13+column)==card){
                    found = 1;
                    printf("%5s of %-
8s%c",wface[column],wsuit[row], card%2==0? '\n':'\t');
                }
            }
    }
```

【例9】有若干计算机图书，请按字母顺序，从小到大输出书名。

解题要求：使用排序函数完成排序，在主函数中进行输入输出。



□ 指针数组

数组中的元素为指针变量。常用于处理多个字符串。

数据类型 * 数组名[数组长度];

```
char * name[SIZE];
```

7.4 指针数组

```
#include<stdio.h>
#include<stdlib.h>
#define SIZE 1000
#define LEN 256
void strSort(char * a[], int n);
int main(void)
{ int n,i;
    char * name[SIZE];
    printf(" input lines number:");    scanf("%d",&n);
    getchar();//读取回车符，否则系统会认为输入的第一个串会是空串
    for(i = 0; i <= n-1; i++){
        name[i] =(char *) malloc(LEN);
        if ( name[i]==NULL)
            printf(" %d string malloc failed\n",i+1);
        else    fgets(name[i],LEN,stdin); //读取字符串
    }
```

7.4 指针数组

```
strSort(name, n); /*n书名按字典序升序排序*/
```

```
printf(" sorted strings :\n");  
for(i = 0; i <= n-1; i++)  
    fputs(name[i],stdout);
```

```
//内存释放,先释放字符串再释放指针数组
```

```
for(i = 0; i <= n-1; i++){  
    free(name[i]);  
    name[i] = NULL;
```

```
}
```

```
return 0;
```

```
}
```

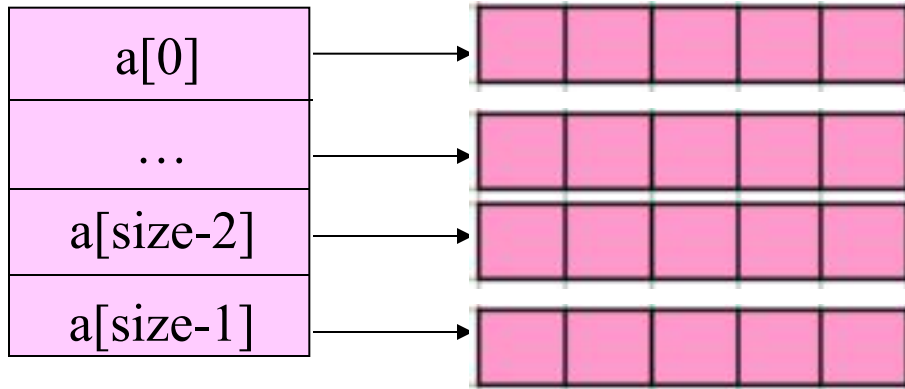
`void sort(char * a[], int n);` 对指针数组中各字符串进行字典排序

`i=0; i<=size-2; i++`

从`a[i]~a[size-1]`指向的字符串中查找最小串，该最小串由`a[min]`指向

交换`a[i]`和`a[min]`的值，使`a[i]`指向最小串

a



1

1

`min=i`//假设`a[i]`指向的串最小

`j=i+1; j<=size-1; j++`

`strcmp(a[j],a[min]<0)`

Y

N

`min=j`

```
void strSort(char * a[], int n){
    char * pt;
    int i, j, min;
    /*使用选择排序，外循环每循环一次,确定a[i]的值*/
    for(i=0; i<n-1; i++){
        min=i;
        for(j=i+1; j<n; j++)
            if(strcmp(a[min],a[j])>0)
                min=j;
        if(min!=i){
            pt=a[min];
            a[min]=a[i];
            a[i]=pt;
        }
    }
}
```

二维数组排序多字符串方案

```
int main(void){
    char input[SIZE][LEN];
    char *name[SIZE]; /* array of pointer variables */
    int ct = 0;        /* input count*/
    int k;              /* output count*/
    printf("Input up to %d lines, and I will sort them.\n",SIZE);
    printf("To stop, press the Enter key at a line's start.\n");
    fgets(input[ct],LEN,stdin);
    while (ct < SIZE && input[ct][0] != '\0'){
        name[ct] = input[ct]; /*set ptrs to strings*/
        ct++;
        fgets(input[ct],LEN,stdin);
    }
    strSort(name,ct);        /* string sorter */
    puts("\nHere's the sorted list:\n");
    for (k = 0; k < ct; k++)
        fputs(name[k],stdout) ;    /* sorted pointers */
    return 0;
}
```


7.5 指向函数的指针

- 什么是函数指针？

是指向函数的指针，值为函数代码在内存中的起始地址。

函数名实际上是该函数代码在内存存储区域的起始地址。

- 定义形式

数据类型 (*指针变量名)(参数类型);

- `int f1(int, int);` //普通函数的原型声明。
- `int * f2(int, int);` /*返回值为指向int数据指针的函数原型*/
- `int (* pf)(int, int);` /*函数指针的定义，可指向返回值为整型，有两个整型参数的函数*/

普通函数调用示例

```
#include <stdio.h>
int max(int,int);
int main(){
    int a,b,c;
    scanf("%d,%d",&a,&b);
    c=max(a,b);//函数名调用可以改为函数指针调用
    printf("a=%d,b=%d,max=%d",a,b,c);
}
int max(int x, int y){
    int z;
    if(x<y) z=x;
    else z=y;
    return(z);
}
```

函数指针使用示例

//将上例的**main**函数改写为

```
#include <stdio.h>
```

```
int max(int,int);
```

```
int main(){
```

```
    int (*p)(int,int);
```

```
    int a,b,c;
```

```
    scanf("%d,%d",&a,&b);
```

```
    p=max;
```

```
    c=(*p)(a,b);//函数名调用改为函数指针调用
```

```
    printf("a=%d,b=%d,max=%d",a,b,c);
```

```
}
```

7.5 使用函数指针的多用途排序程序

先来看一下两个函数的定义：

/*若 $b < a$ ，即 a 、 b 不是以升序排列的，则返回值1，否则返回值0*/

```
int ascending(const int a, const int b)
```

```
{
```

```
    return b < a;
```

```
}
```

/*若 $b > a$ ，即 a 、 b 不是以降序排列的，则返回值1，，否则返回值0*/

```
int decending(const int a, const int b)
```

```
{
```

```
    return b > a;
```

```
}
```

基于以上两个函数的不使用函数指针的多用途排序程序如下：

```
void bubble(int * work, const int n, int order )
{
    int pass,count;
    void swap(int *,int *);

    for(pass=1;pass<=n-1;pass++)
        for(count=0;count<=n-1-pass;count++)
            if ( order==1 )/*如果是要求升序排序*/
                /* 如果work[count]和work[count+1]不以升序排列则交换*/
                if (ascending(work[count],work[count+1]))
                    swap(& work[count],&work[count+1]);
            else/*如果是要求降序排序*/
                /* 如果work[count]和work[count+1]不以降序排列则交换*/
                if (descending(work[count],work[count+1]))
                    swap(& work[count],&work[count+1]);
}
```

使用函数指针的多用途排序程序

```
int main()
{
    .....
    if (order==1){
        bubble(a, n, ascending);/*将函数名作为实参*/
        .....
    }
    else{
        bubble(a, n, descending);
        .....
    }
    .....
}
```

7.5 使用函数指针的多用途排序程序

`bubble(a,n,ascending);` `bubble(a,n,descending);`

```
void bubble(int * work, const int n, int (*  
    compare)(int,int) )  
{  
    int pass,count;  
    void swap(int *,int *);  
  
    for(pass=1;pass<=last;pass++)  
        for(count=0;count<=last-pass;count++)  
            if ((*compare)(work[count],work[count+1]) )  
                swap(& work[count],&work[count+1]);  
}
```

由传递给compare的指向函数的指针来决定是升序还是降序

使用函数指针的排序库函数

```
#include <stdlib.h>
```

```
void qsort( void *buf, size_t num, size_t size,  
int (*compare)(const void *, const void *) );
```

功能：对`buf`指向的数据(包含`num`项,每项的大小为`size`)进行快速排序。如果函数`compare`的第一个参数小于第二个参数，返回负值；如果等于返回零值；如果大于返回正值。函数对`buf`指向的数据按升序排序。

```
void *bsearch( const void *key, const void  
*buf, size_t num, size_t size, int  
(*compare)(const void *, const void *) );
```

功能：函数用折半查找法在从数组元素`buf[0]`到`buf[num-1]`匹配参数`key`。如果函数`compare`的第一个参数小于第二个参数，返回负值；如果等于返回零值；如果大于返回正值。数组`buf`中的元素应以升序排列。函数`bsearch()`的返回值是指向匹配项，如果没有发现匹配项，返回`NULL`。

多字符串qsort调用实例

```
#include<stdlib.h>
#include<stdio.h>
#include<string.h>
int main(void)
{
    char string_array[10][50]={"John", "Jane", "Mary",
    "Rogery", "Dave", "Paul", "Beavis", "Astro", "George",
    "Elroy"};
    /* Sort the list */
    qsort(*string_array,10,50,strcmp);
    /* Search for the item "Elroy" and print it */
    printf("%s",bsearch("Elroy",string_array,10,50,strcmp));
    return 0;
}
```

整数数组qsort调用实例

```
#include<stdlib.h>
#include<stdio.h>
int main(void)
{
    int array[10]={9,8,7,6,5,4,3,2,1,0};
    /* Sort the list */
    qsort(array,10,sizeof(int),compare);
    /* Search for the item "4" and print it */
    int key=4;
    printf("%d",*(int *)bsearch(&key,array,10, sizeof(int),
    compare));
    return 0;
}
int compare(const void *p,const void *q) {
    return *(int *)p-*(int *)q;
}
```

多字符串qsort降序排列

```
#include<stdlib.h>
#include<stdio.h>
int compare(const void *p,const void *q);
int main(void){
    char string_array[10][50]={"John", "Jane", "Mary",
    "Rogery", "Dave", "Paul", "Beavis", "Astro", "George",
    "Elroy"};
    /* Sort the list */
    qsort(string_array,10,50,compare);
    /* Search for the item "Elroy" and print it */
    printf("%s",bsearch("Elroy",string_array,10,50,compare
    ));
}
int compare(const void *p,const void *q) {
    return -1*(strcmp(p,q));
}
```

整数数组qsort降序排序

```
#include<stdlib.h>
#include<stdio.h>
int main(void)
{
    int array[10]={9,8,7,6,5,4,3,2,1,0};
    /* Sort the list */
    qsort(array,10,sizeof(int),compare);
    /* Search for the item "4" and print it */
    int key=4;
    printf("%d",*(int *)bsearch(&key,array,10, sizeof(int),
compare));
    return 0;
}
int compare(const void *p,const void *q) {
    return *(int *)q-*(int *)p;
}
```

7.5 使用函数指针的多用途排序程序

使用函数指针的优点：

使程序易于扩展。

函数func，参数为指向函数的指针p：

```
void func( int (*p)(int,int))
```

指向函数1的指针

指向函数2的指针

将来，可以再定义符合此函数原型的函数n，调用时赋值给p，从而扩展程序的功能

7.5 演示指向函数的指针数组

函数指针常用在菜单驱动的系统：提供用户不同的菜单项，每一个菜单项对应有一个函数来完成其功能。指向每一个函数的指针存放在指针数组中，用户的选择作为数组的下标，根据用户的选择取出对应数组元素中的指针，然后通过该指针来调用函数。

7.5 演示指向函数的指针数组

声明长度为3的数组f, 数组元素的类型是: void (*)(int), 即指向函数的指针

```
int main()
{
    void (*f[3])(int)={function1,function2,function3};
    int choice;

    printf("Enter a number between 0 and 2, 3 to end");
    scanf("%d",&choice);
    while(choice>=0 && choice<3){
        (*f[choice])(choice);/*调用相应的函数*/
        printf("Enter a number between 0 and 2, 3 to end");
        scanf("%d",&choice);
    }
    return 0;
}
```

